

Technical Documentation

LoRa-Based SOS Communication System Prototype

April 30, 2026

1 System Overview

Our device is a portable distress-communication prototype for infrastructure-denied environments. The implemented system is a single-hop transmit-and-display prototype. It consists of a transmitter node, a receiver node, and an operator device connected to the receiver's local Wi-Fi access point.

When SOS mode is active, the transmitter broadcasts a plain-text emergency payload every two seconds over a 433 MHz LoRa link. The receiver captures the most recent packet, stores the message and RSSI in memory, and serves the alert on a browser-accessible page over its ESP32 Wi-Fi soft access point.

1.1 Implemented Node Roles

- **Transmitter node:** Portable ESP32-based node with an SX1278 LoRa radio, NEO-6M GPS module, and active-low manual SOS button.
- **Receiver node:** ESP32-based receive node with an SX1278 LoRa radio, Wi-Fi soft access point, and HTTP server.
- **Operator device:** Phone or other Wi-Fi-capable client that connects to the receiver access point and opens the alert page in a browser.

2 Implemented Functionality

Function	Current implementation
Manual SOS trigger	GPIO 26 is configured with INPUT_PULLUP. The firmware applies a 50 ms software debounce and toggles the internal SOS state on button press.
GPS acquisition	A u-blox NEO-6M GPS module is connected to ESP32 UART2 at 9600 baud and parsed with TinyGPS++.

LoRa payload transmission	The transmitter sends a human-readable ASCII payload containing either latitude and longitude or a no-fix fallback message.
Receiver browser alert	The receiver starts a Wi-Fi soft access point and HTTP server on port 80. The served page auto-refreshes every three seconds.
Single-hop communication	One transmitter communicates directly with one receiver over the configured LoRa link.

3 Hardware Specification

3.1 Materials

No.	Component	Qty.	Notes
1	Ai-Thinker Ra-02 SX1278 LoRa module, 433 MHz	2	One module on the transmitter and one on the receiver.
2	ESP32 30-pin DevKit with CP2102 USB-UART	2	One development board on each side.
3	u-blox NEO-6M GPS module	1	Used only on the transmitter.
4	Ebyte 4 dBi 433 MHz antenna	2	One antenna per LoRa module.
5	SMA-to-U.FL pigtail, 20 cm	2	Connects each external antenna to the radio module.
6	400-point breadboard	2	Breadboard construction on both sides.
7	Female-to-female jumper wire set	1	Shared assembly stock.
8	Male-to-female jumper wire set	1	Shared assembly stock.
9	Tactile push button	1	Manual SOS trigger.
10	3.7 V 1000 mAh LiPo cell	1	Listed for the transmitter.
11	TP4056 charging board	1	Listed for the transmitter.

3.2 Construction Notes

- The prototype is breadboard-based and does not use a custom PCB.
- The current build does not include an enclosure.
- The current build does not include a dedicated external buzzer, OLED, or status LED beyond the indicator LEDs already present on the development modules.
- The demonstrated radio link uses external 4 dBi antennas. Operation without a properly connected antenna is not representative of the demonstrated setup.

3.3 Transmitter Pin Map

ESP32 pin	Module pin	Function
GPIO 5	Ra-02 NSS	SPI chip select.
GPIO 18	Ra-02 SCK	SPI clock.
GPIO 19	Ra-02 MISO	SPI MISO.
GPIO 23	Ra-02 MOSI	SPI MOSI.
GPIO 2	Ra-02 RESET	Radio reset.
GPIO 4	Ra-02 DIO0	TX-done/RX-done interrupt.
3.3 V	Ra-02 VCC	Radio supply; the Ra-02 is not 5 V tolerant.
GND	Ra-02 GND	Common ground.
GPIO 16 (RX2)	NEO-6M TX	GPS-to-ESP32 data at 9600 baud.
GPIO 17 (TX2)	NEO-6M RX	ESP32-to-GPS line; unused by the current code.
3.3 V or breakout VCC input	NEO-6M VCC	GPS module power through the breakout module.
GND	NEO-6M GND	Common ground.
GPIO 26	SOS button	Active-low manual SOS input using INPUT_PULLUP; debounced in software.

3.4 Receiver Pin Map

The receiver uses the same LoRa wiring as the transmitter for GPIO 5, GPIO 18, GPIO 19, GPIO 23, GPIO 2, and GPIO 4. It does not include a GPS module or button input. The receiver ESP32 also hosts the Wi-Fi soft access point and HTTP server.

4 Firmware and Software Architecture

4.1 Transmitter Behaviour

The transmitter firmware performs the following sequence:

1. Initialize serial debugging at 115200 baud.
2. Initialize UART2 to the NEO-6M GPS module at 9600 baud.
3. Configure the SOS input button on GPIO 26 with INPUT_PULLUP.
4. Initialize the SX1278 radio at 433 MHz with the configured spreading factor, bandwidth, coding rate, transmit power, and sync word.
5. Continuously parse GPS sentences with TinyGPS++.
6. Toggle the internal SOS state when the user briefly presses the button.

7. While SOS is active, transmit one payload every 2000 ms.

The current payload format is plain ASCII text and has two possible forms:

```
SOS! Lat: <latitude_dd.dddddd> Lon: <longitude_dd.dddddd>
SOS! GPS: Waiting for fix...
```

4.2 Receiver Behaviour

The receiver firmware performs the following sequence:

1. Initialize the SX1278 radio with the same RF parameters as the transmitter.
2. Start a local Wi-Fi soft access point using redacted demo credentials.
3. Start an HTTP server on port 80.
4. Poll the radio for packets with `LoRa.parsePacket()`.
5. Store the last received message, packet RSSI, and reception time in RAM.
6. Serve a browser-based page that auto-refreshes every three seconds and displays the most recent emergency alert.

The receiver is receive-and-display only. It does not transmit acknowledgement packets, does not clear stale alerts, and does not persist historical records across reboot.

5 Communication Parameters and Packeting

5.1 RF Configuration

Parameter	Value	Source in code
Carrier frequency	433 MHz	<code>LoRa.begin(433E6)</code>
Modulation	LoRa CSS	SX1278 configuration
Transmit power	20 dBm	<code>LoRa.setTxPower(20)</code>
Spreading factor	SF11	<code>LoRa.setSpreadingFactor(11)</code>
Signal bandwidth	62.5 kHz	<code>LoRa.setSignalBandwidth(62.5E3)</code>
Coding rate	4/8	<code>LoRa.setCodingRate4(8)</code>
Sync word	0xBA	<code>LoRa.setSyncWord(0xBA)</code>
Payload encoding	Plain ASCII text	<code>LoRa.print(payload)</code>
Transmit cadence	One message every 2000 ms while SOS is active	<code>txInterval = 2000</code>

5.2 Current Packet Semantics

Field or aspect	Current implementation
Message type	Implicitly encoded by the SOS! prefix.
Latitude and longitude	Included only when TinyGPS++ reports a valid location.
No-fix indicator	Embedded directly as the text fallback GPS: Waiting for fix....
Device identity	Not present.
Sequence number	Not present.
Timestamp	Not present.
Integrity field	Not present.
Encryption or authentication	Not present.

6 Operational Workflow

1. The user briefly presses the SOS button on the transmitter.
2. The transmitter latches into SOS mode and continuously parses GPS data.
3. Every two seconds, the transmitter emits a plain-text LoRa packet carrying either geographic coordinates or a no-fix message.
4. The receiver captures the packet, stores the message and RSSI, and updates the emergency page served by its local Wi-Fi access point.
5. An operator connects a phone to the local Wi-Fi access point and views the emergency alert in a browser.

7 Validation Evidence

The available field evidence supports the following prototype observations:

- An outdoor LoRa SOS transmission was demonstrated from the terrace of LBS Hall of Residence to a receiver located outside SNVH.
- The transmitted GPS coordinates corresponded to the test origin at approximately 22.320905° N, 87.299588° E.
- The receiver UI displayed the message and an example signal strength of approximately -116 dBm.
- The maximum confirmed test range recorded for this revision is 4 km line of sight.

8 Technical Limitations of the Current Prototype

The current build has the following technical limitations:

- The LoRa payload is transmitted in plain ASCII.
- The receiver does not authenticate the sender.
- The receiver does not detect replayed packets.
- The transmitted packet does not include a device identity, sequence number, timestamp, integrity field, or encrypted payload.
- Demo Wi-Fi credentials are hard-coded in the prototype firmware and are redacted in this documentation.
- Packet-loss statistics, button-to-browser latency, current draw, battery endurance, environmental robustness, and controlled non-line-of-sight performance have not been measured.

A Prototype Firmware Listings

A.1 Transmitter Sketch

```
#include <SPI.h>
#include <LoRa.h>
#include <TinyGPS++.h>

#define PIN_LORA_CS 5
#define PIN_LORA_RST 2
#define PIN_LORA_DIO0 4
#define BUTTON_PIN 26
#define RXD2 16
#define TXD2 17
#define LORA_FREQ 433E6

TinyGPSPlus gps;
HardwareSerial GPS_Serial(2);
bool isSOSActive = false;
unsigned long lastTxTime = 0;
const int txInterval = 2000;
int lastButtonState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long debounceDelay = 50;

void sendSOSMessage() {
  String payload = "SOS! ";

  if (gps.location.isValid()) {
    payload += "Lat: " + String(gps.location.lat(), 6);
    payload += " Lon: " + String(gps.location.lng(), 6);
```

```

    } else {
        payload += "GPS: Waiting for fix...";
    }

    LoRa.beginPacket();
    LoRa.print(payload);
    LoRa.endPacket();
    Serial.println("Sent: " + payload);
}

void setup() {
    Serial.begin(115200);
    GPS_Serial.begin(9600, SERIAL_8N1, RXD2, TXD2);
    Serial.println("Transmitter Started.");
    pinMode(BUTTON_PIN, INPUT_PULLUP);

    LoRa.setPins(PIN_LORA_CS, PIN_LORA_RST, PIN_LORA_DIO0);
    if (!LoRa.begin(LORA_FREQ)) {
        Serial.println("Starting LoRa failed!");
        while (1);
    }

    LoRa.setTxPower(20);
    LoRa.setSpreadingFactor(11);
    LoRa.setSignalBandwidth(62.5E3);
    LoRa.setCodingRate4(8);
    LoRa.setSyncWord(0xBA);
    Serial.println("LoRa Initialized.");
}

void loop() {
    while (GPS_Serial.available() > 0) {
        gps.encode(GPS_Serial.read());
    }

    int reading = digitalRead(BUTTON_PIN);

    if (reading != lastButtonState) {
        lastDebounceTime = millis();
    }

    if ((millis() - lastDebounceTime) > debounceDelay) {
        if (reading == LOW && !isSOSActive) {
            isSOSActive = true;
            Serial.println("\n*** SOS ACTIVATED! ***");
            delay(200);
        } else if (reading == LOW && isSOSActive) {
            isSOSActive = false;
            Serial.println("\n*** SOS DEACTIVATED ***");
            delay(200);
        }
    }

    lastButtonState = reading;
}

```

```

    if (isSOSActive && (millis() - lastTxTime > txInterval)) {
        sendSOSMessage();
        lastTxTime = millis();
    }
}

```

A.2 Receiver Sketch

```

#include <SPI.h>
#include <LoRa.h>
#include <WiFi.h>
#include <WebServer.h>

#define PIN_LORA_CS 5
#define PIN_LORA_RST 2
#define PIN_LORA_DIO0 4
#define LORA_FREQ 433E6

const char* ssid = "<REDACTED>";
const char* password = "<REDACTED>";

WebServer server(80);
String lastSOSMessage = "Waiting for signal...";
int lastRSSI = 0;
unsigned long lastMsgTime = 0;
bool messageReceived = false;

String getHTMLPage() {
    String html = "<!DOCTYPE html><html><head>";
    html += "<meta name='viewport' content='width=device-width, initial-scale=1'>";
    html += "<meta http-equiv='refresh' content='3'>";
    html += "<style>";
    html += "body { font-family: Arial, sans-serif; text-align: center; background-color: #222; color: white; margin-top: 30px; }";
    html += ".alert-box { background-color: #d9534f; padding: 25px; border-radius: 15px; display: inline-block; max-width: 90%; box-shadow: 0 4px 8px rgba(0,0,0,0.5); }";
    html += ".waiting-box { background-color: #5bc0de; padding: 20px; border-radius: 15px; display: inline-block; color: black; }";
    html += "h1 { font-size: 35px; margin-top: 0; }";
    html += "p { font-size: 20px; line-height: 1.5; margin: 10px 0; }";
    html += ".time { font-size: 14px; color: #f9f9f9; }";
    html += "</style></head><body>";
    html += "<h2>LoRa SOS Gateway</h2>";

    if (messageReceived) {
        long secondsAgo = (millis() - lastMsgTime) / 1000;
        html += "<div class='alert-box'>";
        html += "<h1>EMERGENCY</h1>";
        html += "<p><strong>GPS Data:</strong><br>" + lastSOSMessage + "</p>";
        html += "<p><strong>Signal Strength:</strong> " + String(lastRSSI) + " dBm</p>";
        html += "<p class='time'>Last received: " + String(secondsAgo) + " seconds ago</p>";
    }
}

```



```

    html += "</div>";
} else {
    html += "<div class='waiting-box'>";
    html += "<h2>System Active</h2>";
    html += "<p>Listening for SOS transmissions...</p>";
    html += "</div>";
}

html += "</body></html>";
return html;
}

void handleRoot() {
    server.send(200, "text/html", getHTMLPage());
}

void setup() {
    Serial.begin(115200);
    LoRa.setPins(PIN_LORA_CS, PIN_LORA_RST, PIN_LORA_DIO0);

    if (!LoRa.begin(LORA_FREQ)) {
        Serial.println("Starting LoRa failed!");
        while (1);
    }

    LoRa.setTxPower(20);
    LoRa.setSpreadingFactor(11);
    LoRa.setSignalBandwidth(62.5E3);
    LoRa.setCodingRate4(8);
    LoRa.setSyncWord(0xBA);

    WiFi.softAP(ssid, password);
    IPAddress myIP = WiFi.softAPIP();
    Serial.print("AP IP address: ");
    Serial.println(myIP);

    server.on("/", handleRoot);
    server.begin();
}

void loop() {
    server.handleClient();

    int packetSize = LoRa.parsePacket();
    if (packetSize) {
        String incomingMessage = "";
        while (LoRa.available()) {
            incomingMessage += (char)LoRa.read();
        }

        lastSOSMessage = incomingMessage;
        lastRSSI = LoRa.packetRssi();
        lastMsgTime = millis();
        messageReceived = true;
    }
}

```

```
    Serial.println("Received: " + incomingMessage + " | RSSI: " + String(lastRSSI));  
  }  
}
```